Sistema de monitoreo de pulso y oxigenación con ESP32 y MAX30102

Cristian Enrique Cruz Fernandez · Samuel Felipe Diaz Galindo · Santiago Reina Socha
Fundación Universitaria de San Gil – UNISANGIL, sede Yopal
Yopal, Casanare, Colombia

***Resumen*—Se presenta el diseño de un oxímetro de pulso educativo basado en un ESP32, un sensor óptico MAX30102 y una pantalla TFT ST7789. El sensor entrega muestras de luz roja e infrarroja por I²C; el microcontrolador mantiene una ventana deslizante de 100 muestras y ejecuta el algoritmo de frecuencia cardíaca y saturación de oxígeno de la biblioteca SparkFun. El firmware también representa la señal fotopletismográfica (PPG), filtra las estimaciones con promedios exponenciales, detecta la ausencia del dedo y transmite una salida compatible con el Serial Plotter. Se documentan el conexionado, el flujo de operación y los bloques principales del programa. No se reportan valores experimentales porque no se suministraron registros de prueba del montaje físico.**

***Palabras clave*—MAX30102, ESP32, fotopletismografía, frecuencia cardíaca, SpO_2, ST7789.**

I. Introducción

La fotopletismografía óptica permite observar cambios periódicos en la luz detectada a medida que varía el volumen sanguíneo periférico. El MAX30102 integra emisores rojo e infrarrojo, un fotodetector y electrónica de adquisición; comunica las muestras mediante una interfaz compatible con I²C [1]. En este proyecto, un ESP32 lee los dos canales y presenta la onda PPG y estimaciones de frecuencia cardíaca y saturación periférica de oxígeno en una TFT ST7789.

El propósito es documentar una implementación reproducible del montaje y explicar cómo se relacionan el sensor, el muestreo, el algoritmo, la interfaz y la salida serie. El dispositivo es un prototipo educativo: sus resultados dependen del módulo, del contacto del dedo y de la calidad de la señal, y no se presentan como mediciones clínicas.

II. Materiales

El montaje emplea tres módulos principales (Tabla I), ilustrados en las Figs. 1 a 3, además de cables de conexión, protoboard y un cable USB para alimentación y programación.

TABLA I
COMPONENTES PRINCIPALES DEL SISTEMA

Componente	Función	Interfaz	Tensión
ESP32 DevKit	Microcontrolador: adquisición, cálculo y control	I²C, SPI	3,3 V (USB 5 V)
MAX30102	Sensor óptico rojo e infrarrojo (PPG)	I²C	3,3 V
TFT ST7789 1,69"	Pantalla 240×280 px a color	SPI	3,3 V

A. Tarjeta de desarrollo ESP32

Es el microcontrolador principal del proyecto. Lee las muestras del sensor MAX30102 por I²C, ejecuta el algoritmo de cálculo y envía la información a la pantalla por SPI.

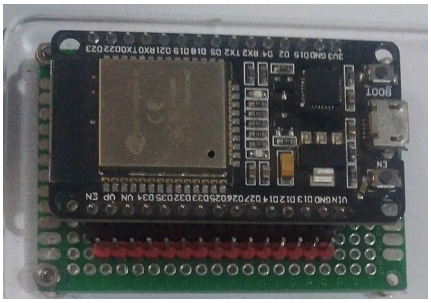


Fig. 1. Tarjeta de desarrollo ESP32 montada sobre una placa de pruebas. Foto: Edwiyanto, CC BY-SA 4.0, vía Wikimedia Commons (recortada) [2].

B. Pantalla TFT ST7789 de 1,69"

Es el dispositivo utilizado para visualizar la onda PPG, la frecuencia cardíaca en BPM y la saturación de oxígeno. Su controlador es el ST7789V2 y se conecta al ESP32 mediante comunicación SPI.



Fig. 2. Módulo LCD IPS de 1,69" (240×280 px, controlador ST7789V2). Imagen: Waveshare [3].

C. Sensor MAX30102

Sensor óptico que mide la frecuencia cardíaca y la saturación de oxígeno (SpO_2) mediante fotopletismografía. Se conecta al ESP32 por I²C y se alimenta con 3,3 V.



Fig. 3. Módulo sensor MAX30102 (pines VIN, SCL, SDA, INT, IRD, RD y GND). Imagen: Circuit.Rocks [4].

III. Objetivos

A. Objetivo general

Implementar y describir un sistema con ESP32 y MAX30102 que adquiera la señal óptica del dedo, estime BPM y SpO_2, y muestre la onda PPG en pantalla y en el Serial Plotter de Arduino.

B. Objetivos específicos

- Configurar el MAX30102 y organizar las muestras en una ventana deslizante de 100 valores compatible con el algoritmo seleccionado.
- Visualizar la señal PPG junto con el estado del sensor y las estimaciones válidas.
- Explicar el cableado, el almacenamiento circular de muestras, el cálculo y la comunicación serie.

- Identificar las condiciones de prueba que deben verificarse antes de interpretar las lecturas.

IV. Versiones del prototipo

Durante el desarrollo se implementaron dos versiones del sistema, con el objetivo de mejorar el rendimiento en la visualización de la señal PPG y de la frecuencia cardíaca en la pantalla TFT (Tabla II).

A. Versión 1

La primera versión cumplía con la adquisición de las señales y su visualización en la pantalla ST7789, pero la actualización era lenta: en cada ciclo se redibujaba la pantalla completa, lo que consumía mucho tiempo de procesamiento del ESP32 a través del bus SPI. Como consecuencia, la señal se veía entrecortada y el valor de BPM se actualizaba con retraso, lo que dificultaba observar el pulso en tiempo real (Fig. 4).

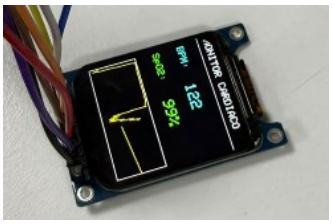


Fig. 4. Versión 1 del prototipo (fuente: elaboración propia).

B. Versión 2

En la segunda versión se optimizó el código: se redujo la cantidad de información enviada a la pantalla, de modo que solo se actualizan las zonas que cambian (el trazo de la señal y el valor numérico) en lugar de redibujar todo el display. Además, se mejoró la gestión del tiempo de lectura de los sensores y se aprovechó mejor la velocidad del bus SPI. Con estos cambios, la señal se muestra de forma más fluida y continua, y la frecuencia cardíaca se actualiza casi en tiempo real (Fig. 5).

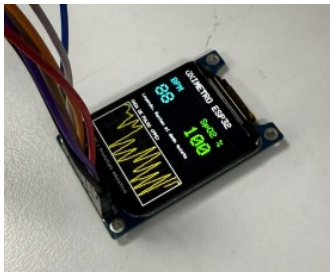


Fig. 5. Versión 2 del prototipo (fuente: elaboración propia).

TABLA II
COMPARACIÓN CUALITATIVA ENTRE VERSIONES

Aspecto	Versión 1	Versión 2
Redibujado	Pantalla completa	Solo zonas que cambian
Carga del bus SPI	Alta	Reducida
Trazo de la señal	Entrecortado	Fluido y continuo
Actualización de BPM	Con retraso	Casi en tiempo real

V. Diagrama de conexiones

La Fig. 6 muestra el cableado de los tres módulos y la Tabla III lista cada conexión. El sensor usa un bus I²C propio (GPIO 25 y 26) y la pantalla usa el bus SPI por hardware del ESP32; ambos módulos se alimentan con 3,3 V.

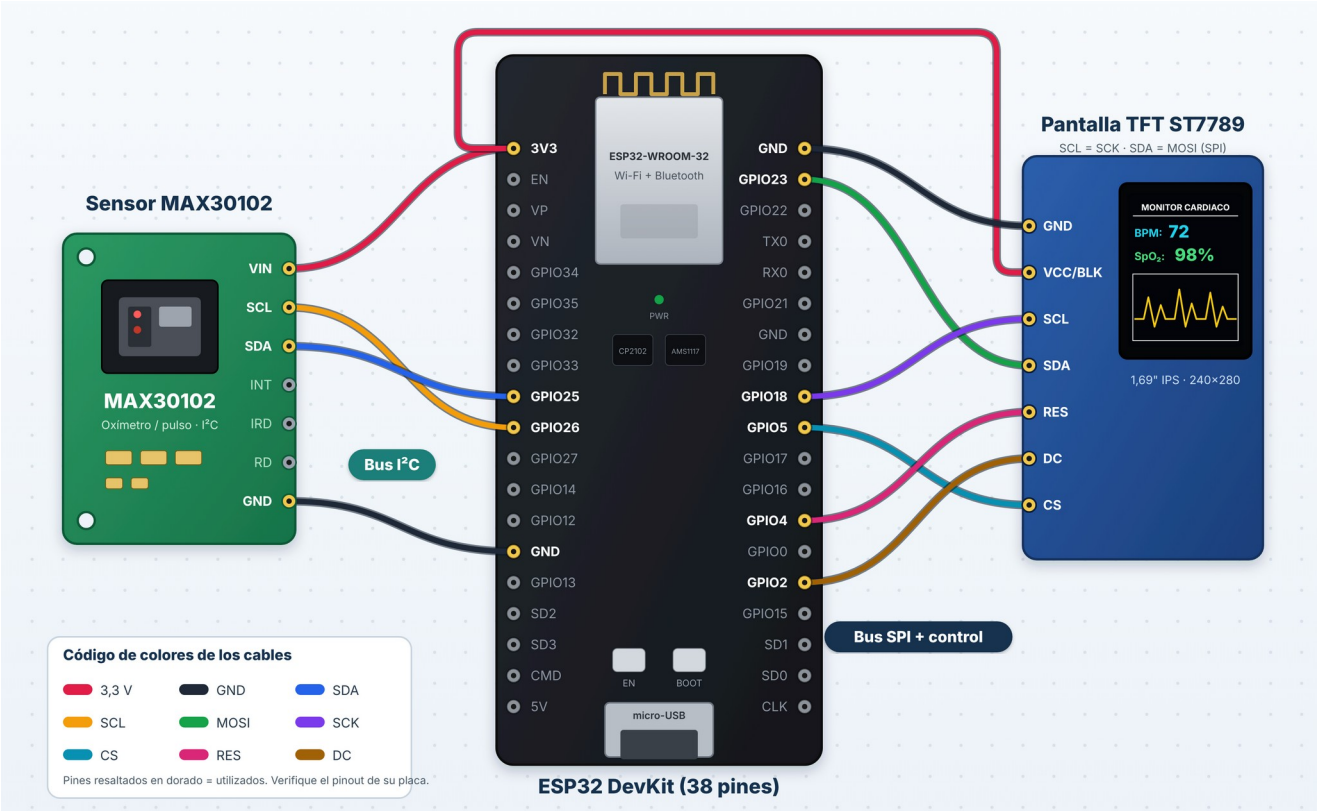


Fig. 6. Diagrama de conexiones entre el ESP32, el sensor MAX30102 y la pantalla ST7789 (fuente: elaboración propia; distribución de pines típica de una ESP32 DevKit de 38 pines).

TABLA III
CONEXIONES ENTRE MÓDULOS Y ESP32

Módulo / señal	Pin ESP32	Interfaz
MAX30102 SDA	GPIO 25	I²C
MAX30102 SCL	GPIO 26	I²C
MAX30102 VIN	3,3 V	Alimentación
MAX30102 GND	GND	Común
TFT SDA / MOSI	GPIO 23	SPI
TFT SCL / SCK	GPIO 18	SPI
TFT CS	GPIO 5	Control
TFT DC	GPIO 2	Control
TFT RES	GPIO 4	Control
TFT VCC / BLK	3,3 V	Alimentación
TFT GND	GND	Común

Notas de conexión

- En la TFT, las etiquetas SDA y SCL corresponden a MOSI y SCK del bus SPI, respectivamente.
- El sensor MAX30102 usa un bus I²C independiente en GPIO 25 y GPIO 26.
- Los pines VCC y BLK de la pantalla se unen a 3,3 V.
- Confirme el pinout impreso en cada módulo antes de energizar el circuito; el diagrama supone el módulo y la placa de expansión indicados en el montaje original.

VI. Diagrama de flujo

La Fig. 7 resume el firmware en cuatro etapas: inicialización (pasos 1 a 4), adquisición (5 a 8), procesamiento (9 a 12) y presentación (13 a 16). Las decisiones detectan un sensor sin respuesta, una muestra que no llega a tiempo y el intervalo de refresco de la pantalla.

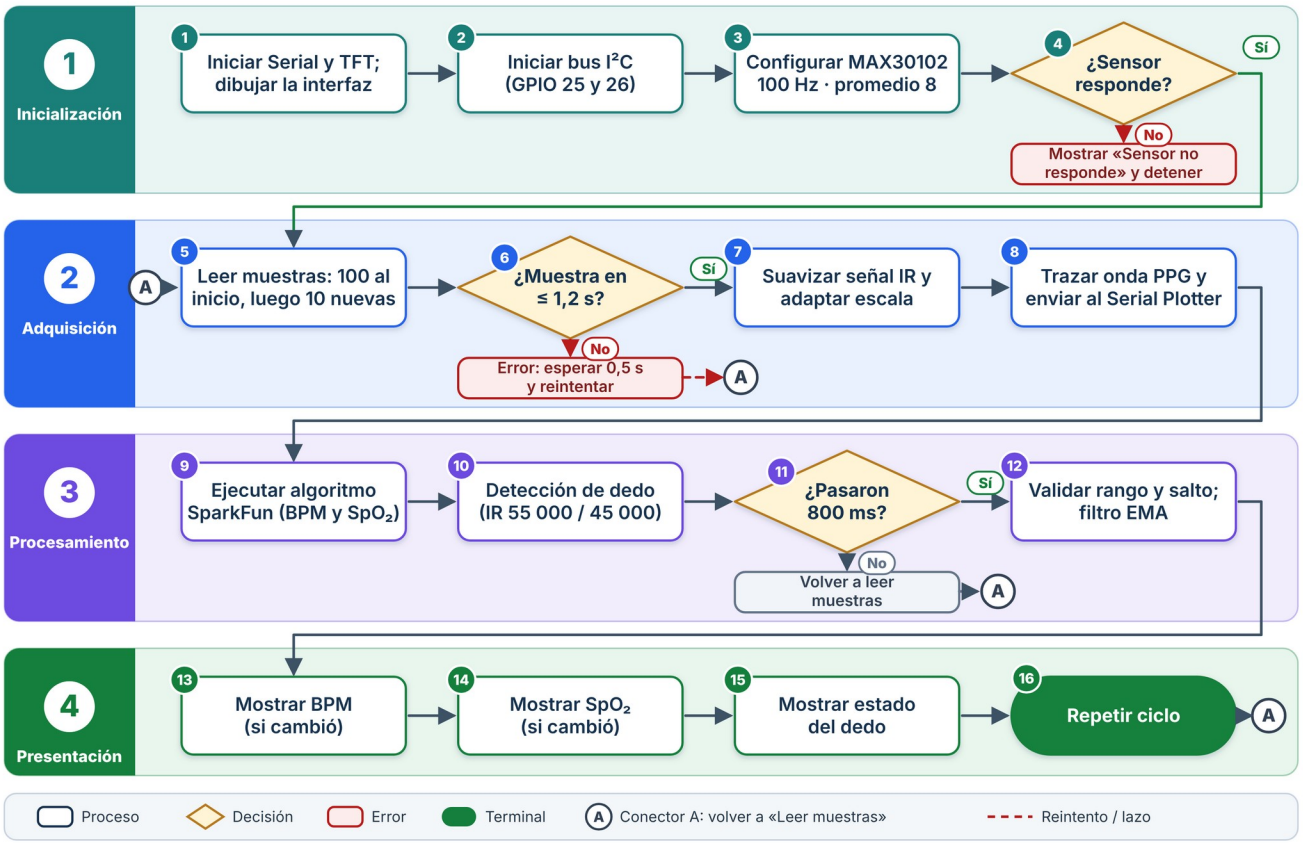


Fig. 7. Diagrama de flujo del firmware, organizado en cuatro etapas y 16 pasos numerados (fuente: elaboración propia).

VII. Configuración y procesamiento

El sensor se configura a 100 muestras/s con promedio de 8 muestras, lo que da una tasa efectiva de 12,5 muestras/s (Tabla IV). El programa conserva una ventana de 100 muestras (≈8 s) y en cada ciclo incorpora 10 nuevas antes de invocar el algoritmo de la biblioteca SparkFun [5]. El ejemplo original de esa biblioteca usa promedio de 4 (25 muestras/s) [6], por lo que conviene validar experimentalmente la configuración elegida. Las estimaciones solo se aceptan si son válidas, caen en un rango fisiológico y no saltan más de lo permitido; luego se suavizan con un promedio exponencial (Tabla V).

TABLA IV
PARÁMETROS DE CONFIGURACIÓN DEL MAX30102

Parámetro	Valor
Brillo del LED	60
Promedio de muestras	8
Modo de LED	2 (rojo + IR)
Frecuencia de muestreo	100 muestras/s
Ancho de pulso	411 µs
Rango del ADC	4096 nA
Tasa efectiva	12,5 muestras/s
Ventana / paso	100 / 10 muestras

TABLA V
LÍMITES, FILTROS Y TIEMPOS DEL FIRMWARE

Magnitud	BPM	SpO_2
Rango aceptado	40–180	70–100 %
Salto máximo	25	6
Factor EMA	0,30	0,25
Factor EMA inicial	0,65	0,55
Detección de dedo (IR)	55 000 / 45 000	—
Refresco de pantalla	800 ms	800 ms
Tiempo límite del sensor	1 200 ms	1 200 ms

VIII. Código explicado

El firmware completo (Arduino, C++) se presenta dividido en bloques funcionales. Usa las bibliotecas Adafruit GFX [7], Adafruit ST7789 [8] y SparkFun MAX3010x [5].

Código 1. Encabezado, conexiones y librerías

Comentario inicial con el cableado y las instrucciones para el Serial Plotter, seguido de las bibliotecas utilizadas (Wire, SPI, Adafruit GFX/ST7789 y SparkFun MAX3010x [5], [7], [8]).

```
/*
  ======================================================================
  Oxímetro ESP32 + MAX30102 + TFT ST7789  —  v3
  BPM, SpO2 y onda de pulsaciones (pantalla + Serial Plotter de Arduino)
  ======================================================================

  CONEXIONES (placa de expansión):
  ------------------------------------------------------------
  PANTALLA (lado derecho)      PLACA
    SCL                         D18
    SDA                         D23
    RES                         D4
    DC                          D2
    CS                          D5
    VCC y BLK                   VCC (3.3V)
    GND                         GND
  ------------------------------------------------------------
  MAX30102 (lado izquierdo)    PLACA
    SDA                         D25
    SCL                         D26
    GND                         GND
    VIN                         3.3V
  ------------------------------------------------------------

  CÓMO VER LAS PULSACIONES EN ARDUINO IDE:
    1. Sube el programa.
    2. Menú Herramientas > Serial Plotter (Ctrl+Shift+L).
    3. Velocidad: 115200 baudios.
    4. Verás 3 líneas:
         Pulso -> la onda de cada latido (escala 0 a 100)
         BPM   -> pulsaciones por minuto
         SpO2  -> oxígeno en sangre (%)
    Si prefieres ver texto, pon MODO_PLOTTER = false y abre el
    Monitor Serie (no uses los dos a la vez).

  LIBRERÍAS: Adafruit GFX, Adafruit ST7789, SparkFun MAX3010x
  ======================================================================
*/

#include <Wire.h>
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7789.h>
#include "MAX30105.h"
#include "spo2_algorithm.h"
```

Código 2. Configuración

Constantes de pines, parámetros del sensor (brillo del LED, promedio, modo, tasa, ancho de pulso y rango del ADC), límites fisiológicos de los filtros, umbrales de detección del dedo y dimensiones de la gráfica.

```
// ============================ CONFIGURACIÓN ============================

// ---- Pantalla TFT (SPI por hardware: SCL = GPIO18, SDA = GPIO23) ----
constexpr int TFT_CS  = 5;
constexpr int TFT_DC  = 2;
constexpr int TFT_RST = 4;
constexpr int TFT_SCREEN_W = 240;
constexpr int TFT_SCREEN_H = 280;

// ---- Sensor MAX30102 (I2C en pines nuevos) ----
constexpr int SDA_PIN = 25;
```

```
constexpr int SCL_PIN = 26;
constexpr byte LED_BRIGHTNESS = 60;
constexpr byte SAMPLE_AVERAGE = 8;
constexpr byte LED_MODE       = 2;
constexpr byte SAMPLE_RATE    = 100;
constexpr int  PULSE_WIDTH    = 411;
constexpr int  ADC_RANGE      = 4096;

// ---- Buffers del algoritmo ----
constexpr int32_t BUFFER_LENGTH   = 100;
constexpr byte    SAMPLES_PER_CYCLE = 10;

// ---- Filtros ----
constexpr int BPM_MIN = 40, BPM_MAX = 180;
constexpr int BPM_MAX_JUMP = 25;
constexpr int SPO2_MIN = 70, SPO2_MAX = 100;
constexpr int SPO2_MAX_JUMP = 6;
constexpr float EMA_ALPHA_BPM  = 0.30f;
constexpr float EMA_ALPHA_SPO2 = 0.25f;
constexpr float EMA_ALPHA_BPM_INICIAL  = 0.65f;
constexpr float EMA_ALPHA_SPO2_INICIAL = 0.55f;
constexpr byte  LECTURAS_CALIBRACION_RAPIDA = 4;

// ---- Detección de dedo (histéresis) ----
constexpr long IR_DEDO_PUESTO  = 55000;
constexpr long IR_DEDO_QUITADO = 45000;

// ---- Tiempos ----
constexpr unsigned long INTERVALO_PANTALLA = 800;
constexpr unsigned long TIMEOUT_SENSOR_MS  = 1200;

// ---- Gráfica en pantalla ----
constexpr int GRAPH_X = 10;
constexpr int GRAPH_Y = 160;
constexpr int GRAPH_WIDTH  = 220;
constexpr int GRAPH_HEIGHT = 90;
constexpr uint16_t GRAPH_BG = ST77XX_BLACK;
constexpr float GRAPH_SUAVIZADO = 0.35f;
constexpr float ADAPT_DECAY = 0.0015f;
constexpr float ADAPT_DECAY_RAPIDO = 0.02f;
constexpr int   MUESTRAS_CALIBRACION_RAPIDA = 120;

// ---- Salida por puerto serie ----
// true  = formato para Serial Plotter (onda + BPM + SpO2)
// false = texto legible para Monitor Serie
constexpr bool MODO_PLOTTER = true;
```

Código 3. Objetos y estado global

Instancias de la pantalla y del sensor, búferes de las señales roja e infrarroja, resultados del algoritmo, valores filtrados y variables de control, junto con los prototipos de las funciones.

```
// ============================ OBJETOS Y ESTADO ============================

Adafruit_ST7789 tft = Adafruit_ST7789(TFT_CS, TFT_DC, TFT_RST);
MAX30105 particleSensor;

uint32_t irBuffer[BUFFER_LENGTH];
uint32_t redBuffer[BUFFER_LENGTH];

int32_t spo2;
int8_t  validSPO2;
int32_t heartRate;
int8_t  validHeartRate;

int   lastBPM  = -1;
int   lastSPO2 = -1;
float bpmFiltrado  = -1;
float spo2Filtrado = -1;

bool primeraVez = true;
bool dedoPuesto  = false;

unsigned long ultimaActualizacionPantalla = 0;

int   graphBuffer[GRAPH_WIDTH];
int   graphIndex = 0;
float irSuavizado = 0;
bool  irSuavizadoInicializado = false;
float irMinAdapt = 30000;
float irMaxAdapt = 120000;

int muestrasDesdeDedoPuesto = 0;
byte lecturasBPMDesdeDedoPuesto  = 0;
byte lecturasSPO2DesdeDedoPuesto = 0;

// ---- Prototipos ----
bool inicializarSensor();
bool leerNuevasMuestras();
void imprimirDebug();
```

```
void actualizarDeteccionDedo(long irValue);
void dibujarInterfaz();
void mostrarErrorSensor();
void mostrarResultados();
void mostrarEstadoDedo();
void actualizarGrafica(uint32_t irValueRaw);
void enviarAlPlotter(float onda);
```

Código 4. Inicialización (setup)

Arranca el puerto serie, la pantalla y el bus I²C; inicializa el MAX30102 y, si no responde, muestra un error y detiene el programa.

```
// ============================ SETUP ============================

void setup() {
  Serial.begin(115200);

  tft.init(TFT_SCREEN_W, TFT_SCREEN_H);
  tft.setRotation(0);
  tft.fillScreen(ST77XX_BLACK);
  dibujarInterfaz();

  Wire.begin(SDA_PIN, SCL_PIN);

  if (!inicializarSensor()) {
    mostrarErrorSensor();
    while (true) { delay(1000); }
  }

  for (int i = 0; i < GRAPH_WIDTH; i++) {
    graphBuffer[i] = GRAPH_Y + GRAPH_HEIGHT / 2;
  }
}

bool inicializarSensor() {
  if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
    return false;
  }
  particleSensor.setup(LED_BRIGHTNESS, SAMPLE_AVERAGE, LED_MODE, SAMPLE_RATE, PULSE_WIDTH,
ADC_RANGE);
  return true;
}
```

Código 5. Bucle principal (loop)

Lee nuevas muestras, ejecuta el algoritmo de BPM y SpO_2, actualiza la detección de dedo y refresca la pantalla cada 800 ms. El búfer se desplaza 10 posiciones por ciclo y se envía una línea al Serial Plotter por cada muestra.

```
// ============================ LOOP ============================

void loop() {
  if (!leerNuevasMuestras()) {
    mostrarErrorSensor();
    delay(500);
    return;
  }

  maxim_heart_rate_and_oxygen_saturation(irBuffer, BUFFER_LENGTH, redBuffer,
                                         &spo2, &validSPO2, &heartRate, &validHeartRate);

  if (!MODO_PLOTTER) imprimirDebug();

  actualizarDeteccionDedo(irBuffer[BUFFER_LENGTH - 1]);

  unsigned long ahora = millis();
  if (ahora - ultimaActualizacionPantalla >= INTERVALO_PANTALLA) {
    ultimaActualizacionPantalla = ahora;
    mostrarResultados();
    mostrarEstadoDedo();
  }
}

bool leerNuevasMuestras() {
  auto esperarMuestra = [&]() -> bool {
    unsigned long inicioEspera = millis();
    while (particleSensor.available() == false) {
      particleSensor.check();
      if (millis() - inicioEspera > TIMEOUT_SENSOR_MS) return false;
    }
    return true;
  };

  if (primeraVez) {
    for (byte i = 0; i < BUFFER_LENGTH; i++) {
      if (!esperarMuestra()) return false;
      redBuffer[i] = particleSensor.getRed();
      irBuffer[i]  = particleSensor.getIR();
      particleSensor.nextSample();
      actualizarGrafica(irBuffer[i]);
    }
    primeraVez = false;
```

```
  } else {
    for (byte i = SAMPLES_PER_CYCLE; i < BUFFER_LENGTH; i++) {
      redBuffer[i - SAMPLES_PER_CYCLE] = redBuffer[i];
      irBuffer[i - SAMPLES_PER_CYCLE]  = irBuffer[i];
    }
    for (byte i = BUFFER_LENGTH - SAMPLES_PER_CYCLE; i < BUFFER_LENGTH; i++) {
      if (!esperarMuestra()) return false;
      redBuffer[i] = particleSensor.getRed();
      irBuffer[i]  = particleSensor.getIR();
      particleSensor.nextSample();
      actualizarGrafica(irBuffer[i]);
    }
  }
  return true;
}

// Texto para el Monitor Serie (solo si MODO_PLOTTER = false)
void imprimirDebug() {
  Serial.print("IR="); Serial.print(irBuffer[BUFFER_LENGTH - 1]);
  Serial.print(" BPM="); Serial.print(heartRate);
  Serial.print(" validBPM="); Serial.print(validHeartRate);
  Serial.print(" SpO2="); Serial.print(spo2);
  Serial.print(" validSpO2="); Serial.println(validSPO2);
}

// Formato "etiqueta:valor,etiqueta:valor" que entiende el Serial Plotter.
// Se envía una línea por cada muestra del sensor, así la onda se ve fluida.
void enviarAlPlotter(float onda) {
  if (!dedoPuesto) onda = 0;                        // sin dedo: línea plana
  int bpm  = (dedoPuesto && lastBPM  > 0) ? lastBPM  : 0;
  int sp   = (dedoPuesto && lastSPO2 > 0) ? lastSPO2 : 0;
  Serial.print("Pulso:"); Serial.print(onda, 1);
  Serial.print(",BPM:");  Serial.print(bpm);
  Serial.print(",SpO2:"); Serial.println(sp);
}
```

Código 6. Detección de dedo

Usa histéresis sobre la intensidad infrarroja (55 000 para detectar y 45 000 para soltar) y reinicia los filtros cuando se retira el dedo.

```
// ============================== DETECCIÓN DE DEDO ==============================

void actualizarDeteccionDedo(long irValue) {
  if (dedoPuesto && irValue < IR_DEDO_QUITADO) {
    dedoPuesto = false;
    lastBPM = -1;
    lastSPO2 = -1;
    bpmFiltrado = -1;
    spo2Filtrado = -1;
    lecturasBPMDesdeDedoPuesto = 0;
    lecturasSPO2DesdeDedoPuesto = 0;
    tft.fillRect(90, 60, 130, 30, ST77XX_BLACK);
    tft.fillRect(110, 110, 110, 30, ST77XX_BLACK);
  } else if (!dedoPuesto && irValue > IR_DEDO_PUESTO) {
    dedoPuesto = true;
    irMinAdapt = irValue - 2000;
    irMaxAdapt = irValue + 2000;
    muestrasDesdeDedoPuesto = 0;
    lecturasBPMDesdeDedoPuesto = 0;
    lecturasSPO2DesdeDedoPuesto = 0;
  }
}
```

Código 7. Interfaz y resultados en pantalla

Dibuja la interfaz fija y actualiza solo los valores que cambian. BPM y SpO_2 solo se aceptan si son válidos, fisiológicos y estables, y se suavizan con un promedio exponencial.

```
// ============================== INTERFAZ ==============================

void dibujarInterfaz() {
  tft.fillScreen(ST77XX_BLACK);

  tft.setTextColor(ST77XX_WHITE);
  tft.setTextSize(2);
  tft.setCursor(10, 5);
  tft.println("MONITOR CARDIACO");

  tft.drawFastHLine(0, 30, tft.width(), ST77XX_WHITE);

  tft.setTextSize(2);
  tft.setCursor(10, 60);
  tft.setTextColor(ST77XX_CYAN);
  tft.print("BPM:");

  tft.setCursor(10, 110);
  tft.setTextColor(ST77XX_GREEN);
  tft.print("SpO2:");

  tft.drawRect(GRAPH_X - 2, GRAPH_Y - 2, GRAPH_WIDTH + 4, GRAPH_HEIGHT + 4, ST77XX_WHITE);
}
```

```

void mostrarErrorSensor() {
  tft.fillRect(0, 0, tft.width(), 40, ST77XX_BLACK);
  tft.setCursor(10, 10);
  tft.setTextColor(ST77XX_RED);
  tft.setTextSize(2);
  tft.println("Sensor no");
  tft.println("responde!");
  if (!MODO_PLOTTER) Serial.println("MAX30102: sin respuesta. Revisa cableado/alimentación.");
}

void mostrarResultados() {
  if (!dedoPuesto) return;

  // -------- BPM --------
  bool bpmFisiologico = (heartRate >= BPM_MIN && heartRate <= BPM_MAX);
  bool bpmEstable = (lastBPM == -1) || (abs(heartRate - lastBPM) <= BPM_MAX_JUMP);

  if (validHeartRate && bpmFisiologico && bpmEstable) {
    float alphaBPM = (lecturasBPMDesdeDedoPuesto < LECTURAS_CALIBRACION_RAPIDA)
                       ? EMA_ALPHA_BPM_INICIAL
                       : EMA_ALPHA_BPM;
    bpmFiltrado = (bpmFiltrado < 0)
                    ? heartRate
                    : (alphaBPM * heartRate + (1 - alphaBPM) * bpmFiltrado);
    if (lecturasBPMDesdeDedoPuesto < 255) lecturasBPMDesdeDedoPuesto++;
    int bpmMostrado = round(bpmFiltrado);
    if (bpmMostrado != lastBPM) {
      tft.fillRect(90, 60, 130, 30, ST77XX_BLACK);
      tft.setCursor(90, 60);
      tft.setTextSize(3);
      tft.setTextColor(ST77XX_CYAN);
      tft.print(bpmMostrado);
      lastBPM = bpmMostrado;
    }
  }

  // -------- SpO2 --------
  bool spo2Fisiologico = (spo2 >= SPO2_MIN && spo2 <= SPO2_MAX);
  bool spo2Estable = (lastSPO2 == -1) || (abs(spo2 - lastSPO2) <= SPO2_MAX_JUMP);

  if (validSPO2 && spo2Fisiologico && spo2Estable) {
    float alphaSPO2 = (lecturasSPO2DesdeDedoPuesto < LECTURAS_CALIBRACION_RAPIDA)
                        ? EMA_ALPHA_SPO2_INICIAL
                        : EMA_ALPHA_SPO2;
    spo2Filtrado = (spo2Filtrado < 0)
                     ? spo2
                     : (alphaSPO2 * spo2 + (1 - alphaSPO2) * spo2Filtrado);
    if (lecturasSPO2DesdeDedoPuesto < 255) lecturasSPO2DesdeDedoPuesto++;
    int spo2Mostrado = round(spo2Filtrado);
    if (spo2Mostrado != lastSPO2) {
      tft.fillRect(110, 110, 110, 30, ST77XX_BLACK);
      tft.setCursor(110, 110);
      tft.setTextSize(3);
      tft.setTextColor(ST77XX_GREEN);
      tft.print(spo2Mostrado);
      tft.print("%");
      lastSPO2 = spo2Mostrado;
    }
  }
}

void mostrarEstadoDedo() {
  tft.fillRect(0, 140, tft.width(), 15, ST77XX_BLACK);
  if (!dedoPuesto) {
    tft.setCursor(10, 140);
    tft.setTextSize(1);
    tft.setTextColor(ST77XX_RED);
    tft.print("Coloca el dedo sobre el sensor");
  }
}
```

Código 8. Gráfica de la señal PPG

Suaviza la señal infrarroja, adapta dinámicamente sus límites mínimo y máximo, normaliza la onda a 0–100 para el Serial Plotter y traza la curva en pantalla con un barrido circular.

```
// ============================ GRÁFICA ============================

void actualizarGrafica(uint32_t irValueRaw) {
  if (!irSuavizadoInicializado) {
    irSuavizado = irValueRaw;
    irSuavizadoInicializado = true;
  } else {
    irSuavizado = GRAPH_SUAVIZADO * irValueRaw + (1 - GRAPH_SUAVIZADO) * irSuavizado;
  }

  float decayActual = (muestrasDesdeDedoPuesto < MUESTRAS_CALIBRACION_RAPIDA)
                        ? ADAPT_DECAY_RAPIDO
                        : ADAPT_DECAY;
```

```
  if (muestrasDesdeDedoPuesto < MUESTRAS_CALIBRACION_RAPIDA) muestrasDesdeDedoPuesto++;

  if (irSuavizado < irMinAdapt) irMinAdapt = irSuavizado;
  if (irSuavizado > irMaxAdapt) irMaxAdapt = irSuavizado;
  irMinAdapt += (irSuavizado - irMinAdapt) * decayActual;
  irMaxAdapt -= (irMaxAdapt - irSuavizado) * decayActual;
  if (irMaxAdapt - irMinAdapt < 1000) irMaxAdapt = irMinAdapt + 1000;

  // Onda normalizada 0..100 para el Serial Plotter
  if (MODO_PLOTTER) {
    float onda = 100.0f * (irSuavizado - irMinAdapt) / (irMaxAdapt - irMinAdapt);
    onda = constrain(onda, 0.0f, 100.0f);
    enviarAlPlotter(onda);
  }

  int y = map((long)irSuavizado, (long)irMinAdapt, (long)irMaxAdapt,
              GRAPH_Y + GRAPH_HEIGHT, GRAPH_Y);
  y = constrain(y, GRAPH_Y, GRAPH_Y + GRAPH_HEIGHT);

  int prevIndex = (graphIndex == 0) ? GRAPH_WIDTH - 1 : graphIndex - 1;

  tft.drawFastVLine(GRAPH_X + graphIndex, GRAPH_Y, GRAPH_HEIGHT, GRAPH_BG);

  tft.drawLine(GRAPH_X + prevIndex, graphBuffer[prevIndex],
               GRAPH_X + graphIndex, y, ST77XX_YELLOW);

  graphBuffer[graphIndex] = y;
  graphIndex = (graphIndex + 1) % GRAPH_WIDTH;

  int barridoIndex = (graphIndex + 2) % GRAPH_WIDTH;
  tft.drawFastVLine(GRAPH_X + barridoIndex, GRAPH_Y, GRAPH_HEIGHT, GRAPH_BG);
}
```

IX. Conclusiones

El sistema documentado integra ESP32, MAX30102 y ST7789 para adquirir luz roja e infrarroja, procesar una ventana deslizante de 100 muestras y mostrar estimaciones de BPM, SpO_2 y la onda PPG. El diagrama identifica el bus I²C del sensor y el SPI de la pantalla; la explicación del código describe el procesamiento, la validación, el suavizado y la recuperación ante fallos. Se trata de una plataforma académica: hacen falta pruebas físicas documentadas y comparación con un instrumento de referencia antes de valorar la exactitud de las lecturas.

Uso de inteligencia artificial

Para la elaboración de este informe se utilizó el asistente de inteligencia artificial Claude (Anthropic) [9] como apoyo en la revisión de la redacción, la reorganización y el formato del documento, la selección de imágenes con su atribución y la elaboración de los diagramas de las Figs. 6 y 7. Los autores revisaron el contenido y asumen la responsabilidad total del trabajo.

Referencias

[1] Analog Devices, “MAX30102: High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health,” hoja de datos, Rev. 1, 2018. [En línea]. Disponible: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX30102.pdf

[2] Edwiyanto, “ESP32 Dev Board.jpg,” Wikimedia Commons, 28 dic. 2024. Licencia CC BY-SA 4.0. [En línea]. Disponible: https://commons.wikimedia.org/wiki/File:ESP32_Dev_Board.jpg

[3] Waveshare, “1.69inch LCD Display Module, 240×280, SPI, ST7789V2,” página de producto. [En línea]. Disponible: https://www.waveshare.com/1.69inch-lcd-module.htm (consultado el 9 oct. 2026).

[4] Circuit.Rocks, “Pulse Oximeter & Heart Rate Sensor Breakout MAX30102,” página de producto. [En línea]. Disponible: https://circuit.rocks/products/pulse-oximeter-heart-rate-sensor-breakout-max30102 (consultado el 9 oct. 2026).

[5] SparkFun Electronics, “SparkFun MAX3010x Sensor Library,” repositorio y documentación. [En línea]. Disponible: https://github.com/sparkfun/SparkFun_MAX3010x_Sensor_Library

[6] SparkFun Electronics, “Example8_SPO2.ino,” SparkFun MAX3010x Sensor Library. [En línea]. Disponible: https://github.com/sparkfun/SparkFun_MAX3010x_Sensor_Library/blob/master/examples/Example8_SPO2/Example8_SPO2.ino

[7] Adafruit Industries, “Adafruit GFX Library,” repositorio. [En línea]. Disponible: https://github.com/adafruit/Adafruit-GFX-Library

[8] Adafruit Industries, “Adafruit ST7735 and ST7789 Library,” repositorio. [En línea]. Disponible: https://github.com/adafruit/Adafruit-ST7735-Library

[9] Anthropic, “Claude,” asistente de inteligencia artificial, modelo Claude Sonnet 5.5, 2026. [En línea]. Disponible: https://claude.ai